AI ASSISTED CODING

ASSIGNMENT-11.3

NAME:DIDDI KRISHNA SATHWIKA

ROLL NO:2303A52451

BATCH-50

Task 1: Smart Contact Manager (Arrays & Linked Lists)

Scenario

SR University's student club requires a simple Contact Manager Application to store members' names and phone numbers. The system should support efficient addition, searching, and deletion of contacts.

Tasks

1. Implement the contact manager using arrays (lists).

2. Implement the same functionality using a linked list for dynamic memory allocation.

3. Implement the following operations in both approaches:

o Add a contact

o Search for a contact

o Delete a contact

4. Use GitHub Copilot to assist in generating search and delete methods.

5. Compare array vs. linked list approaches with respect to:

o Insertion efficiency

o Deletion efficiency

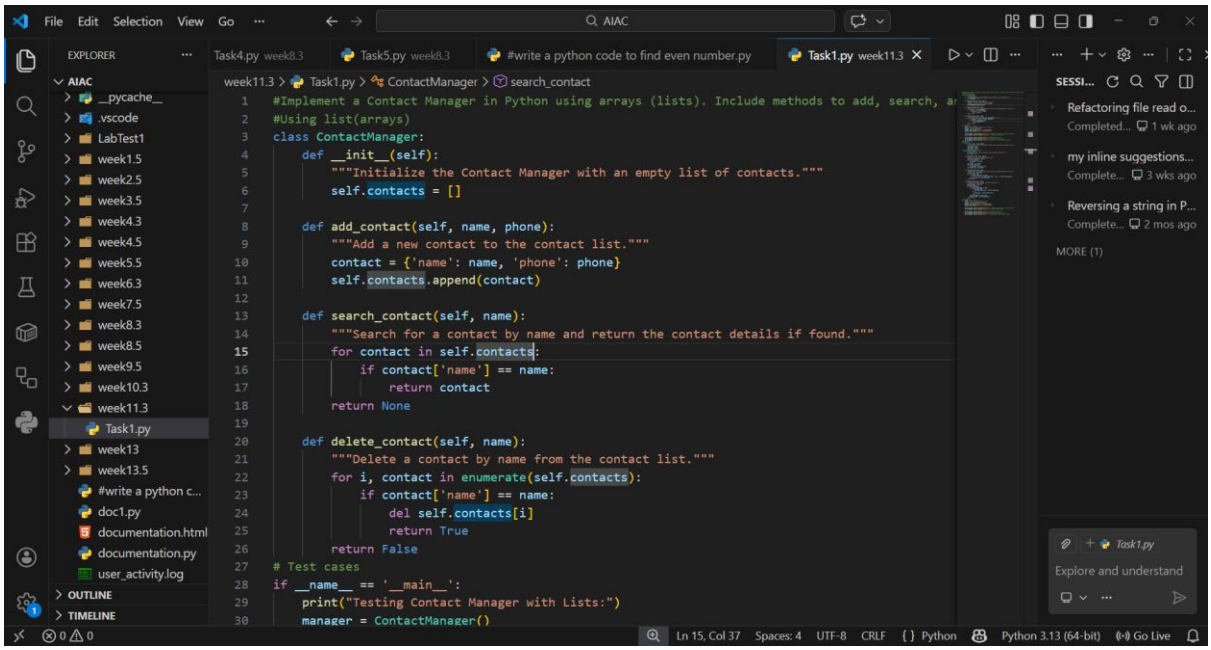
Expected Outcome

- **Two working implementations (array-based and linked-list-based).**
- **A brief comparison explaining performance differences.**

PROMPT:

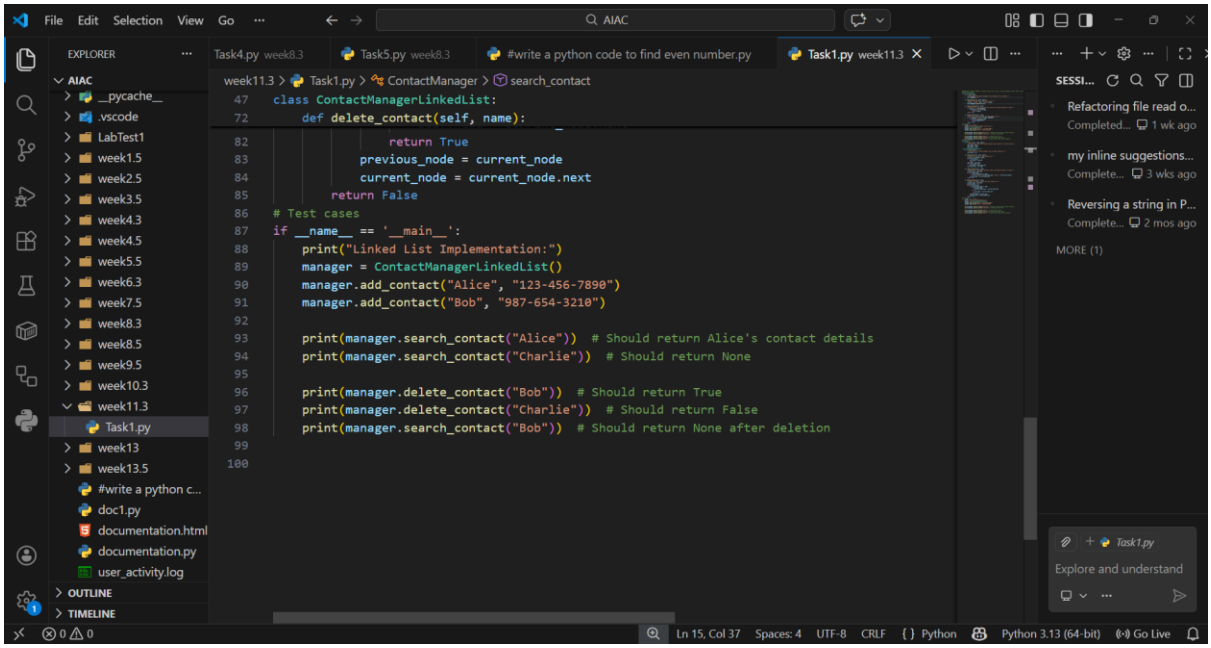
"Implement a Contact Manager in Python using arrays (lists). Include methods to add, search, and delete contacts. Then,perform the same task using linkedlist."

CODE:



```python
    manager = ContactManager()
    manager.add_contact("Alice", "123-456-7890")
    manager.add_contact("Bob", "987-654-3210")

    print(manager.search_contact("Alice"))  # Should return Alice's contact details
    print(manager.search_contact("Charlie"))  # Should return None

    print(manager.delete_contact("Bob"))  # Should return True
    print(manager.delete_contact("Charlie"))  # Should return False
    print(manager.search_contact("Bob"))  # Should return None after deletion
#Using linked list
class Node:
    def __init__(self, name, phone):
        """Initialize a node with contact details and a pointer to the next node."""
        self.name = name
        self.phone = phone
        self.next = None
class ContactManagerLinkedList:
    def __init__(self):
        """Initialize the Contact Manager with an empty linked list."""
        self.head = None

    def add_contact(self, name, phone):
        """Add a new contact to the linked list."""
        new_node = Node(name, phone)
        if not self.head:
            self.head = new_node
            return
        last_node = self.head
        while last_node.next:
```

```python
class ContactManagerLinkedList:
    def add_contact(self, name, phone):
            last_node = last_node.next
        last_node.next = new_node

    def search_contact(self, name):
        """Search for a contact by name and return the contact details if found."""
        current_node = self.head
        while current_node:
            if current_node.name == name:
                return {'name': current_node.name, 'phone': current_node.phone}
            current_node = current_node.next
        return None

    def delete_contact(self, name):
        """Delete a contact by name from the linked list."""
        current_node = self.head
        previous_node = None
        while current_node:
            if current_node.name == name:
                if previous_node:
                    previous_node.next = current_node.next
                else:
                    self.head = current_node.next
                return True
            previous_node = current_node
            current_node = current_node.next
        return False
# Test cases
```



OUTPUT:

```
ython.exe c:/Users/krish/OneDrive/Desktop/AIAC/week11.3/Task1.py
Testing Contact Manager with Lists:
{'name': 'Alice', 'phone': '123-456-7890'}
None
True
False
None
Linked List Implementation:
{'name': 'Alice', 'phone': '123-456-7890'}
None
True
False
None
PS C:\Users\krish\OneDrive\Desktop\AIAC>
```

Explanation:

The contact manager demonstrates how arrays and linked lists handle data storage differently. Arrays provide simple indexing but require shifting elements during deletion, which can be inefficient. Linked lists, on the other hand, allow dynamic memory allocation and make insertion and deletion faster since no shifting is needed. Both approaches have linear search time, but linked lists are more suitable when frequent modifications are expected.

Task 2: Library Book Search System (Queues & Priority Queues)

Scenario

The SRU Library manages book borrow requests. Students and faculty submit requests, but faculty requests must be prioritized over student requests.

Tasks

1. Implement a Queue (FIFO) to manage book requests.

2. Extend the system to a Priority Queue, prioritizing faculty requests.

3. Use GitHub Copilot to assist in generating:

o enqueue() method

o dequeue() method

4. Test the system with a mix of student and faculty requests.

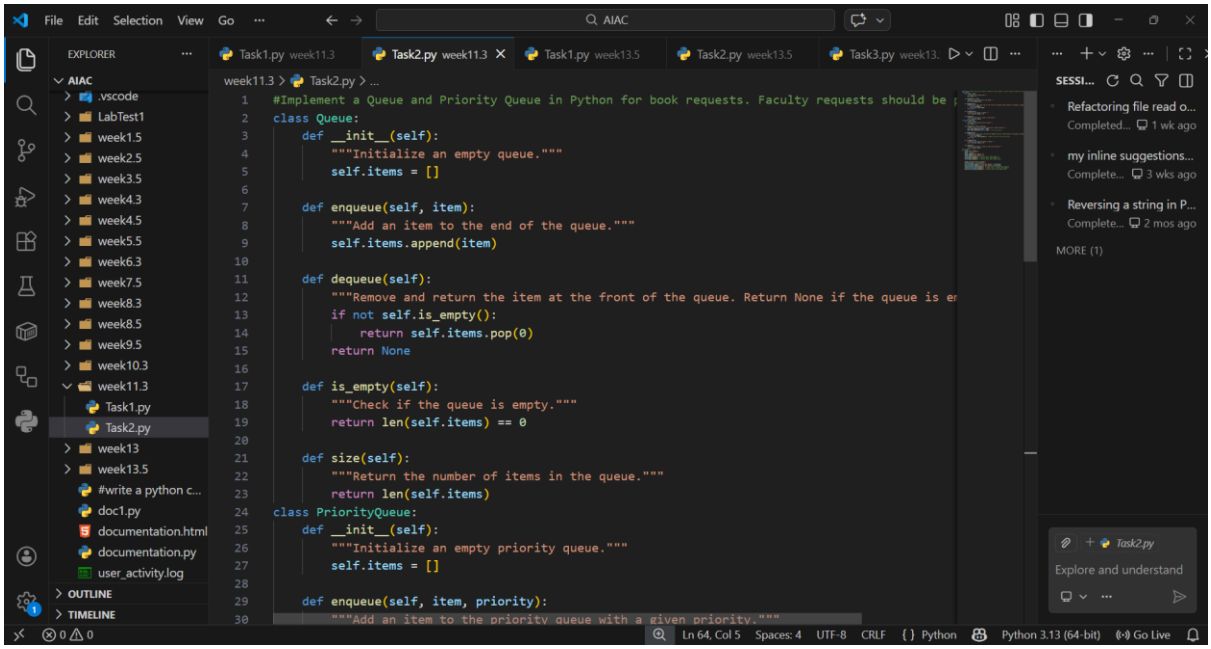
Expected Outcome

- **Working queue and priority queue implementations.**
- **Correct prioritization of faculty requests.**

PROMPT:

"Implement a Queue and Priority Queue in Python for book requests. Faculty requests should be prioritized."

CODE:



```python
class PriorityQueue:
    def enqueue(self, item, priority):
        self.items.append((priority, item))
        self.items.sort(key=lambda x: x[0])  # Sort by priority

    def dequeue(self):
        """Remove and return the item with the highest priority. Return None if the queue is er
        if not self.is_empty():
            return self.items.pop(0)[1]  # Return the item, not the priority
        return None

    def is_empty(self):
        """Check if the priority queue is empty."""
        return len(self.items) == 0

    def size(self):
        """Return the number of items in the priority queue."""
        return len(self.items)
# Test cases
if __name__ == '__main__':
    print("Testing Queue:")
    queue = Queue()
    queue.enqueue("Book Request 1")
    queue.enqueue("Book Request 2")
    print(queue.dequeue())  # Should return "Book Request 1"
    print(queue.dequeue())  # Should return "Book Request 2"
    print(queue.dequeue())  # Should return None (empty queue)

    print("\nTesting Priority Queue:")
```

```python
    queue = Queue()
    queue.enqueue("Book Request 1")
    queue.enqueue("Book Request 2")
    print(queue.dequeue())  # Should return "Book Request 1"
    print(queue.dequeue())  # Should return "Book Request 2"
    print(queue.dequeue())  # Should return None (empty queue)

    print("\nTesting Priority Queue:")
    priority_queue = PriorityQueue()
    priority_queue.enqueue("Faculty Book Request", priority=1)
    priority_queue.enqueue("Student Book Request", priority=2)
    print(priority_queue.dequeue())  # Should return "Faculty Book Request"
    print(priority_queue.dequeue())  # Should return "Student Book Request"
    print(priority_queue.dequeue())  # Should return None (empty queue)
```

OUTPUT:

```
ython.exe c:/Users/krish/OneDrive/Desktop/AIAC/week11.3/Task2.py
Testing Queue:
Book Request 1
Book Request 2
None

Testing Priority Queue:
Faculty Book Request
Student Book Request
None
PS C:\Users\krish\OneDrive\Desktop\AIAC>
```

Explanation:

The queue-based system ensures requests are processed in the order they arrive, following the FIFO principle. However, faculty requests need prioritization, which is achieved using a priority queue. By maintaining separate queues for faculty and students, the system ensures faculty requests are always served first. This highlights how priority queues extend basic queue functionality to handle real-world scheduling needs.

Task 3: Emergency Help Desk (Stack Implementation)

Scenario

SR University's IT Help Desk receives technical support tickets from students and staff. While tickets are received sequentially, issue escalation follows a Last-In, First-Out (LIFO) approach.

Tasks

1. Implement a Stack to manage support tickets.

2. Provide the following operations:

o push(ticket)

o pop()

o peek()

3. Simulate at least five tickets being raised and resolved.

4. Use GitHub Copilot to suggest additional stack operations such as:

o Checking whether the stack is empty

o Checking whether the stack is full (if applicable)

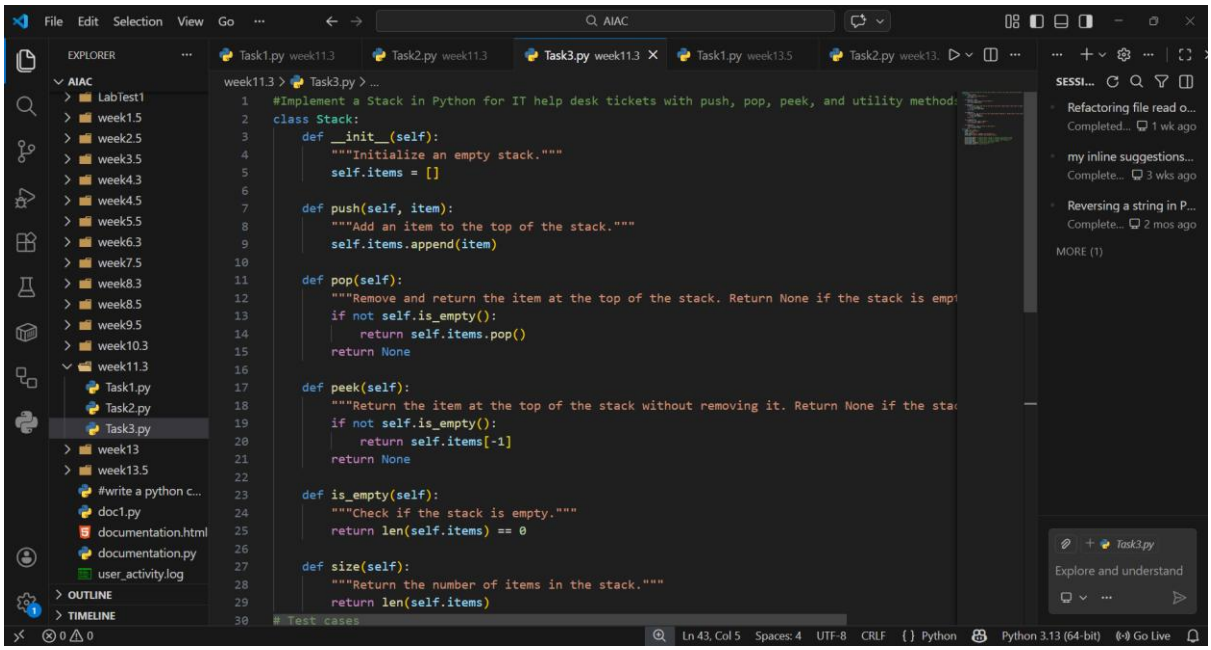
Expected Outcome

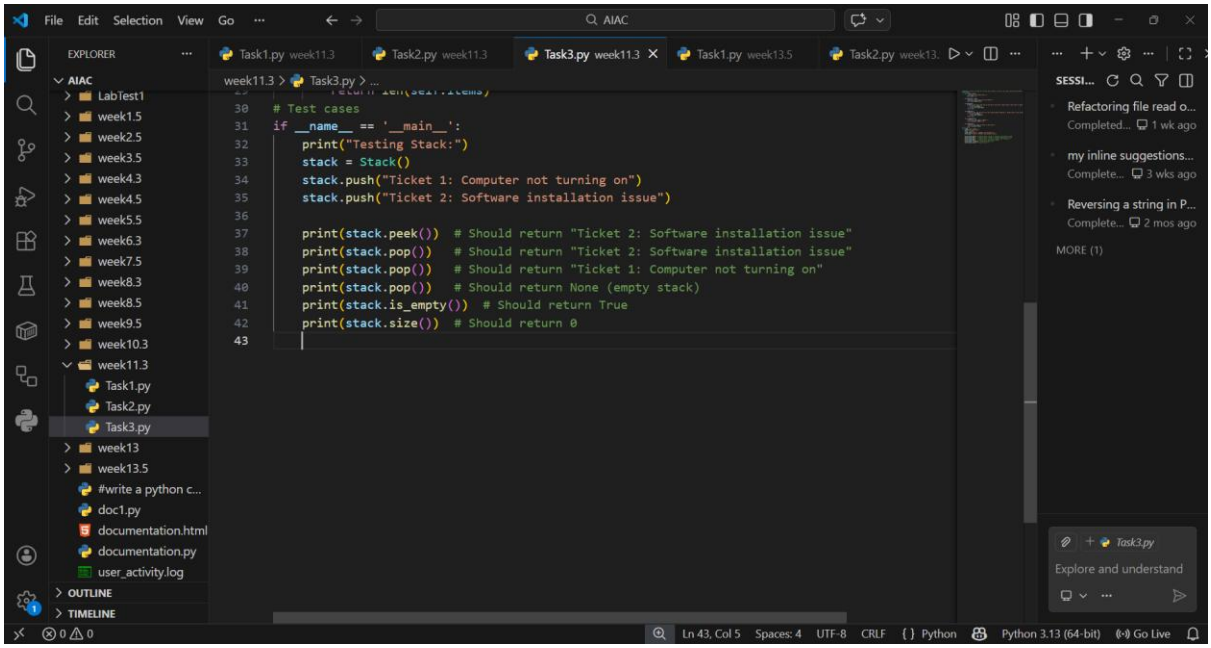
- **Functional stack-based ticket management system.**
- **Clear demonstration of LIFO behavior.**

PROMPT:

“Implement a Stack in Python for IT help desk tickets with push, pop, peek, and utility methods.”

CODE:





OUTPUT:

```
PS C:\Users\krish\OneDrive\Desktop\AIAC> & C:\Users\krish\AppData\Local\Programs\Python\Python313\p
ython.exe c:/Users/krish/OneDrive/Desktop/AIAC/week11.3/Task3.py
Testing Stack:
Ticket 2: Software installation issue
Ticket 2: Software installation issue
Ticket 1: Computer not turning on
None
True
0
PS C:\Users\krish\OneDrive\Desktop\AIAC>
```

Explanation:

The stack implementation models ticket escalation using the LIFO principle, where the most recent issue is resolved first. This mirrors real-world scenarios where urgent or newly raised problems take precedence. Operations like push, pop, and peek demonstrate core stack behavior, while utility methods such as checking emptiness or size enhance usability. The simulation with multiple tickets clearly shows the escalation process in action.

Task 4: Hash Table

Objective

To implement a Hash Table and understand collision handling.

Task Description

Use AI to generate a hash table with:

- **Insert**
- **Search**
- **Delete**

Starter Code

class HashTable:

pass

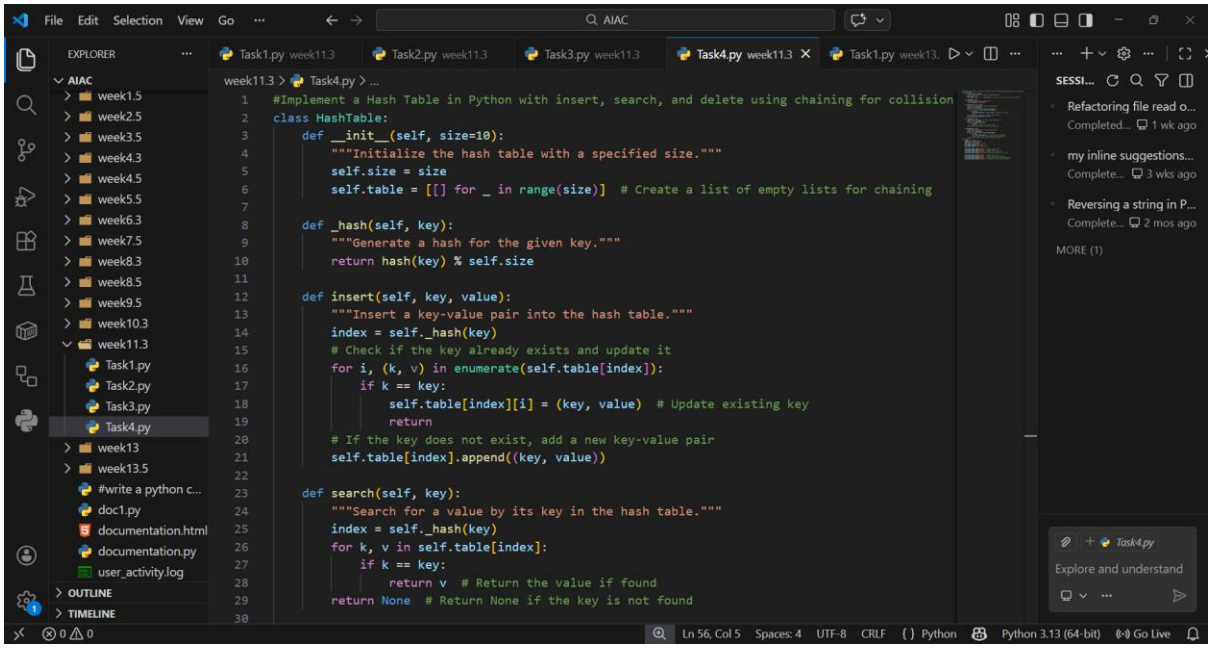
Expected Outcome

- **Collision handling using chaining**
- **Well-commented methods**

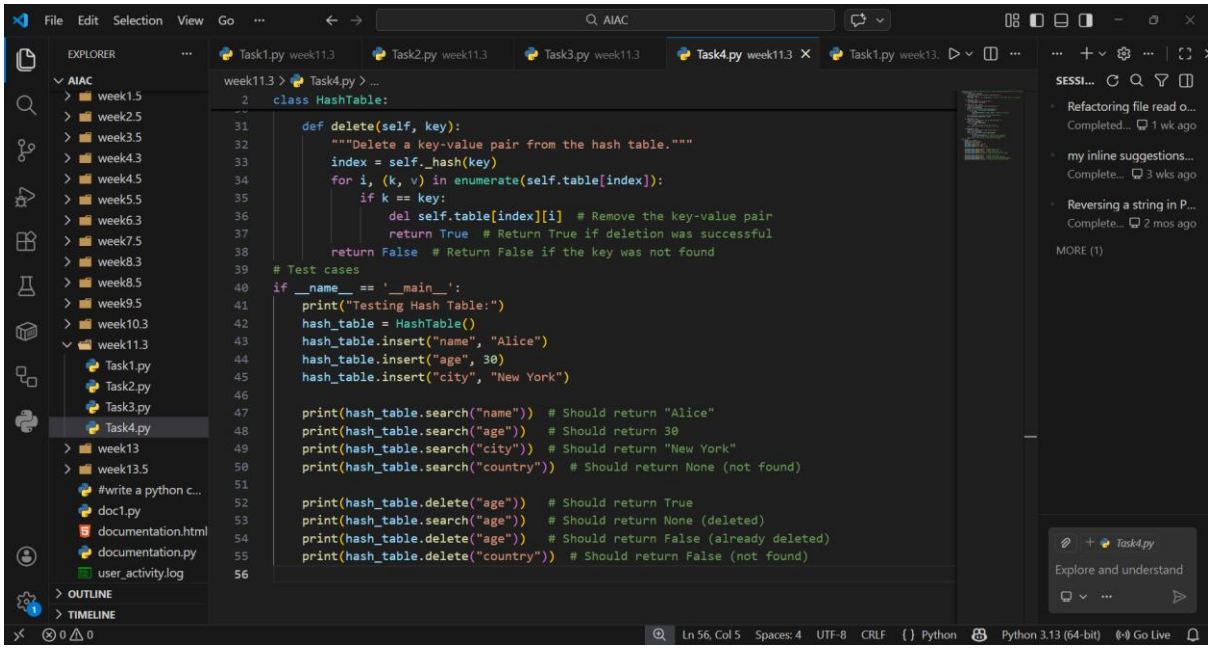
PROMPT:

“Implement a Hash Table in Python with insert, search, and delete using chaining for collision handling.”

CODE:



```python
#Implement a Hash Table in Python with insert, search, and delete using chaining for collision
class HashTable:
    def __init__(self, size=10):
        """Initialize the hash table with a specified size."""
        self.size = size
        self.table = [[] for _ in range(size)]  # Create a list of empty lists for chaining

    def _hash(self, key):
        """Generate a hash for the given key."""
        return hash(key) % self.size

    def insert(self, key, value):
        """Insert a key-value pair into the hash table."""
        index = self._hash(key)
        # Check if the key already exists and update it
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                self.table[index][i] = (key, value)  # Update existing key
                return
        # If the key does not exist, add a new key-value pair
        self.table[index].append((key, value))

    def search(self, key):
        """Search for a value by its key in the hash table."""
        index = self._hash(key)
        for k, v in self.table[index]:
            if k == key:
                return v  # Return the value if found
        return None  # Return None if the key is not found
```



```python
class HashTable:

    def delete(self, key):
        """Delete a key-value pair from the hash table."""
        index = self._hash(key)
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                del self.table[index][i]  # Remove the key-value pair
                return True  # Return True if deletion was successful
        return False  # Return False if the key was not found
# Test cases
if __name__ == '__main__':
    print("Testing Hash Table:")
    hash_table = HashTable()
    hash_table.insert("name", "Alice")
    hash_table.insert("age", 30)
    hash_table.insert("city", "New York")

    print(hash_table.search("name"))  # Should return "Alice"
    print(hash_table.search("age"))   # Should return 30
    print(hash_table.search("city"))  # Should return "New York"
    print(hash_table.search("country"))  # Should return None (not found)

    print(hash_table.delete("age"))   # Should return True
    print(hash_table.search("age"))   # Should return None (deleted)
    print(hash_table.delete("age"))   # Should return False (already deleted)
    print(hash_table.delete("country"))  # Should return False (not found)
```

OUTPUT:

```
ython.exe c:/Users/krish/OneDrive/Desktop/AIAC/week11.3/Task4.py
Testing Hash Table:
Alice
30
New York
None
True
None
False
False
PS C:\Users\krish\OneDrive\Desktop\AIAC>
```

Explanation:

The hash table provides efficient key-based access to data, making insertions and searches faster compared to linear structures. Collisions are inevitable when multiple keys map to the same index, and chaining with linked lists is used to handle them. This ensures that even in collision scenarios, data can be stored and retrieved reliably. The implementation highlights how hashing improves performance in large datasets.

Task 5: Real-Time Application Challenge

Scenario

Design a Campus Resource Management System with the following features:

- **Student Attendance Tracking**
- **Event Registration System**
- **Library Book Borrowing**
- **Bus Scheduling System**
- **Cafeteria Order Queue**

Student Tasks

1. Choose the most appropriate data structure for each feature.

2. Justify your choice in 2–3 sentences.

3. Implement one selected feature using AI-assisted code generation.

Expected Outcome

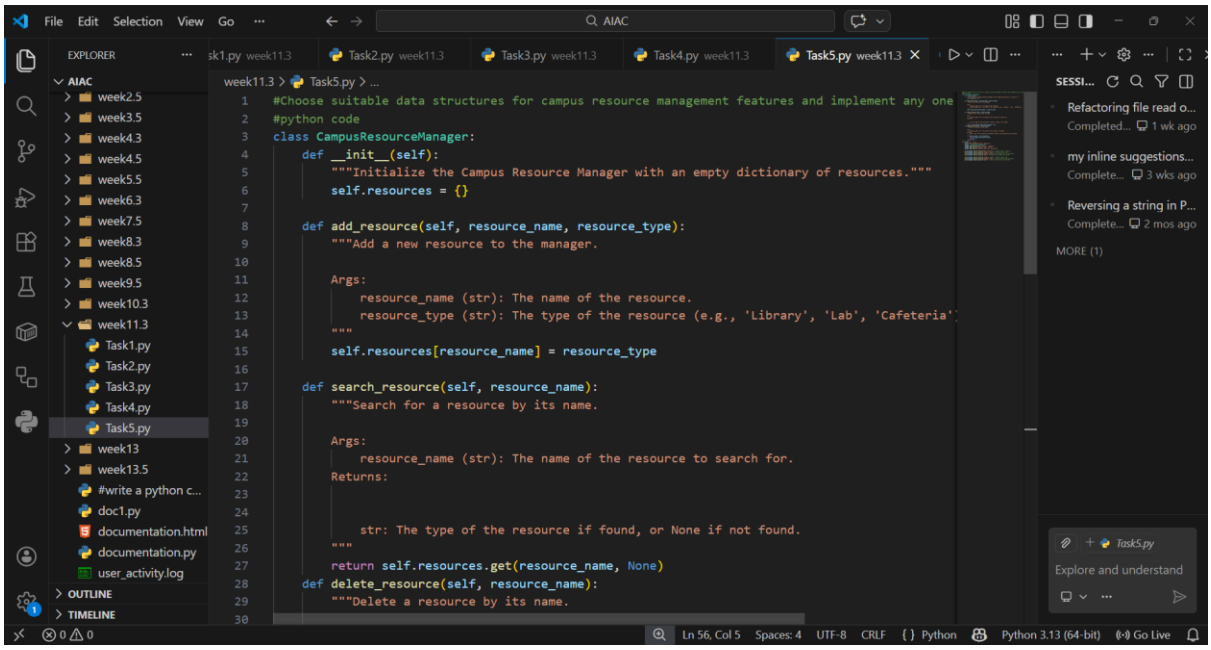
- **Mapping table: Feature → Data Structure → Justification**
- **One fully working Python implementation**

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.

PROMPT:

"Choose suitable data structures for campus resource management features and implement one feature in Python."

CODE:



```python
class CampusResourceManager:
    def delete_resource(self, resource_name):

        Args:
            resource_name (str): The name of the resource to delete.
        Returns:
            bool: True if the resource was deleted, False if the resource was not found.
        """
        if resource_name in self.resources:
            del self.resources[resource_name]
            return True
        return False
# Test cases
if __name__ == '__main__':
    print("Testing Campus Resource Manager:")
    manager = CampusResourceManager()
    manager.add_resource("Main Library", "Library")
    manager.add_resource("Computer Lab", "Lab")
    manager.add_resource("Campus Cafeteria", "Cafeteria")

    print(manager.search_resource("Main Library"))  # Should return "Library"
    print(manager.search_resource("Computer Lab"))  # Should return "Lab"
    print(manager.search_resource("Campus Cafeteria"))  # Should return "Cafeteria"
    print(manager.search_resource("Gym"))  # Should return None (not found)

    print(manager.delete_resource("Computer Lab"))  # Should return True
    print(manager.search_resource("Computer Lab"))  # Should return None (deleted)
    print(manager.delete_resource("Gym"))  # Should return False (not found)
```

OUTPUT:

```
Testing Campus Resource Manager:
Library
Lab
Cafeteria
None
True
None
False
PS C:\Users\krish\OneDrive\Desktop\AIAC>
```

Explanation:

Different campus features require different data structures based on their operational needs. Attendance tracking benefits from hash tables for quick lookups, while queues manage borrowing and cafeteria orders in FIFO order. Linked lists suit bus scheduling due to dynamic route changes, and arrays handle event registrations efficiently. This mapping shows how choosing the right data structure ensures optimal performance for each real-world scenario.